

JAVA DESKTOP APPLICATION

Mini Calculator App

Technical Presentation

A robust, event-driven desktop calculator engineered with Java Swing and AWT. Features real-time calculation history, single-digit erase precision, keyboard shortcuts, and full cross-version bytecode compatibility.

CORE PLATFORM

Java (JDK 8 – 21+)

GUI FRAMEWORK

Java Swing / AWT

ARCHITECTURE

**Event-Driven State
Engine**

INTERFACES

Modern GUI & CLI Mode

System Objectives & Core Highlights

Delivering an intuitive, lightweight desktop application with zero third-party dependencies



Dual Interface Design

Offers two distinct user experiences:

- ✓ **Swing** Sleek dark-mode interface with high-**GUI**: contrast buttons and responsive hover states.
- ✓ **CLI** Lightweight, terminal-based **Mode**: interactive loop for terminal environments.



Precision Editing & Erase

Solves common typing mistakes without resetting state:

- ✓ **Single-Digit Erase** ($\boxtimes$): Backspaces the last typed digit sequentially.
- ✓ **Full Clear** (C): Resets operands, operator, and history labels completely.
- ✓ **Sign Toggle** ($\pm$): Inverts sign with instant visual feedback.



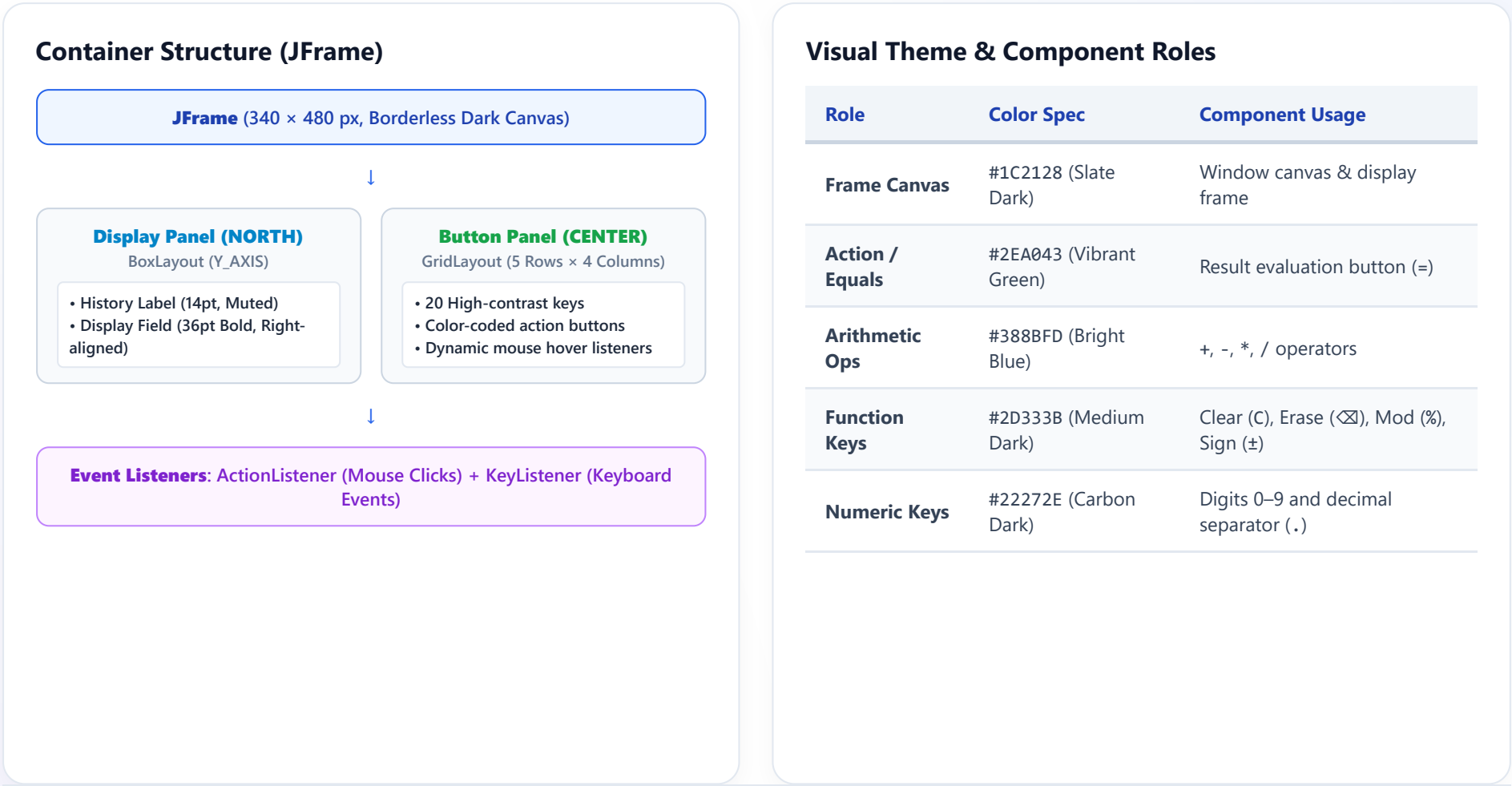
Seamless Keyboard Input

Full physical keyboard synchronization:

- ✓ Type numbers directly from the Numpad or top row.
- ✓ Operators $+, -, *, /$ bound to matching (keys.
- ✓ Enter triggers calculation; Backspace erases a single digit.

User Interface & Component Hierarchy

Constructed using hierarchical Swing containers and modern visual design principles



Single-Digit Erase (⌫) Technical Implementation

Handling complex input boundary conditions, string mutations, and state synchronization

Backspace Logic (Calculator.java)

```
private void handleBackspace() { String text =
displayField.getText(); // 1. Handle error state or zero if
(text.startsWith("Error") || text.equals("0")) {
displayField.setText("0"); startNewNumber = true; return; } //
2. Multi-character deletion if (text.length() > 1) { String
next = text.substring(0, text.length() - 1); // If only
negative sign or blank remains if (next.equals("-") ||
next.equals("-0")) { displayField.setText("0"); startNewNumber
= true; } else { displayField.setText(next); startNewNumber =
false; } } else { // 3. Single digit remaining -> reset to "0"
displayField.setText("0"); startNewNumber = true; } }
```

Defensive Edge-Case Safeguards

Current Buffer	Action	Resulting Buffer	Internal State
"5432"	Press ⌫	"543"	Active input continues
"7"	Press ⌫	"0"	Resets to fresh number ready
"-5"	Press ⌫	"0"	Prevents invalid stray "-"
"12.5"	Press ⌫	"12."	Valid decimal stream kept
"Error: Div by 0"	Press ⌫	"0"	Recovers safely to clean state

Font Compatibility Note: The ⌫ symbol uses Segoe UI Symbol / SansSerif fallback font rendering to guarantee clear visual display across all Windows, macOS, and Linux JVMs without missing glyph boxes.

Cross-Version Compatibility Solution

Diagnosing and resolving Java Runtime version disparities across enterprise environments



The Issue: Runtime Mismatch

When executing the app initially, a Windows error popup stated:

"A Java Exception has occurred."

```
UnsupportedClassVersionError: Calculator has been compiled by a more recent version of Java Runtime (class file version 65.0), this version only recognizes up to 52.0
```

- ✓ **Compiler (javac):**Microsoft OpenJDK 21 (bytecode version 65)
- ✓ **Runtime (java):**Oracle Java 8 on Windows PATH (bytecode version 52)



The Engineering Solution

We implemented a permanent, multi-tiered cross-compatibility strategy:

Step 1: Target Bytecode Release

```
javac --release 8 Calculator.java
```

Forces the Java 21 compiler to emit clean Java 8 (version 52.0) bytecode.

Step 2: Smart Launcher Script

Updated run_gui.bat to autodetect modern JDK paths and fall back to the system environment safely.

Interactive Keymap & Control Reference

Complete mapping of mouse actions and physical keyboard bindings

Category	Button Label	Keyboard Shortcut	Functional Description
Numeric Input	0 – 9	0 – 9 (or Numpad)	Appends digit to current operand buffer
Decimal	.	.	Appends fractional decimal (guards against multiple dots)
Single Erase	<ⓧ	Backspace	Erases the trailing digit from current number
Full Reset	C	Esc	Clears all calculations, history, and active buffers
Arithmetic	+, -, *, /	+ - * /	Commits first operand and sets pending arithmetic operation
Percentage	%	%	Divides current active value by 100 instantly
Sign Inversion	±	Click only	Toggles positive and negative sign
Calculate	=	Enter or =	Evaluates the expression and presents formatted result

Project Summary & Future Roadmap

Key takeaways, competencies demonstrated, and potential expansions



Core Competencies Demonstrated

- ✓ **GUI Architecture:** Component hierarchy, custom styling, border layouts, and dynamic event handling in pure Java.
- ✓ **Robust Edge-Case Handling:** Division by zero protection, decimal formatters, string bounds safety.
- ✓ **Defensive Engineering:** Cross-version JVM bytecode target optimization and auto-detecting batch launchers.
- ✓ **Dual Interface Delivery:** Production-ready Swing GUI alongside terminal CLI alternative.



Future Expansion Roadmap

- ✓ **Scientific Functions:** Trigonometry (sin, cos, tan), Logarithms, Square root ($\sqrt{}$), Exponentials.
- ✓ **Memory Registers:** Standard memory operations (MC, MR, M+, M-).
- ✓ **Calculation History Log:** Expandable drawer to view, export, and re-insert previous calculations.
- ✓ **Theme Customizer:** Light mode, high-contrast mode, and custom accent selections.